# TCP Sockets (Part IV)

## CEN 223 – Internet Communication

Assoc. Prof. Dr. Fatih ABUT
Department of Computer Engineering
Çukurova University

# Client Server Communication (1)

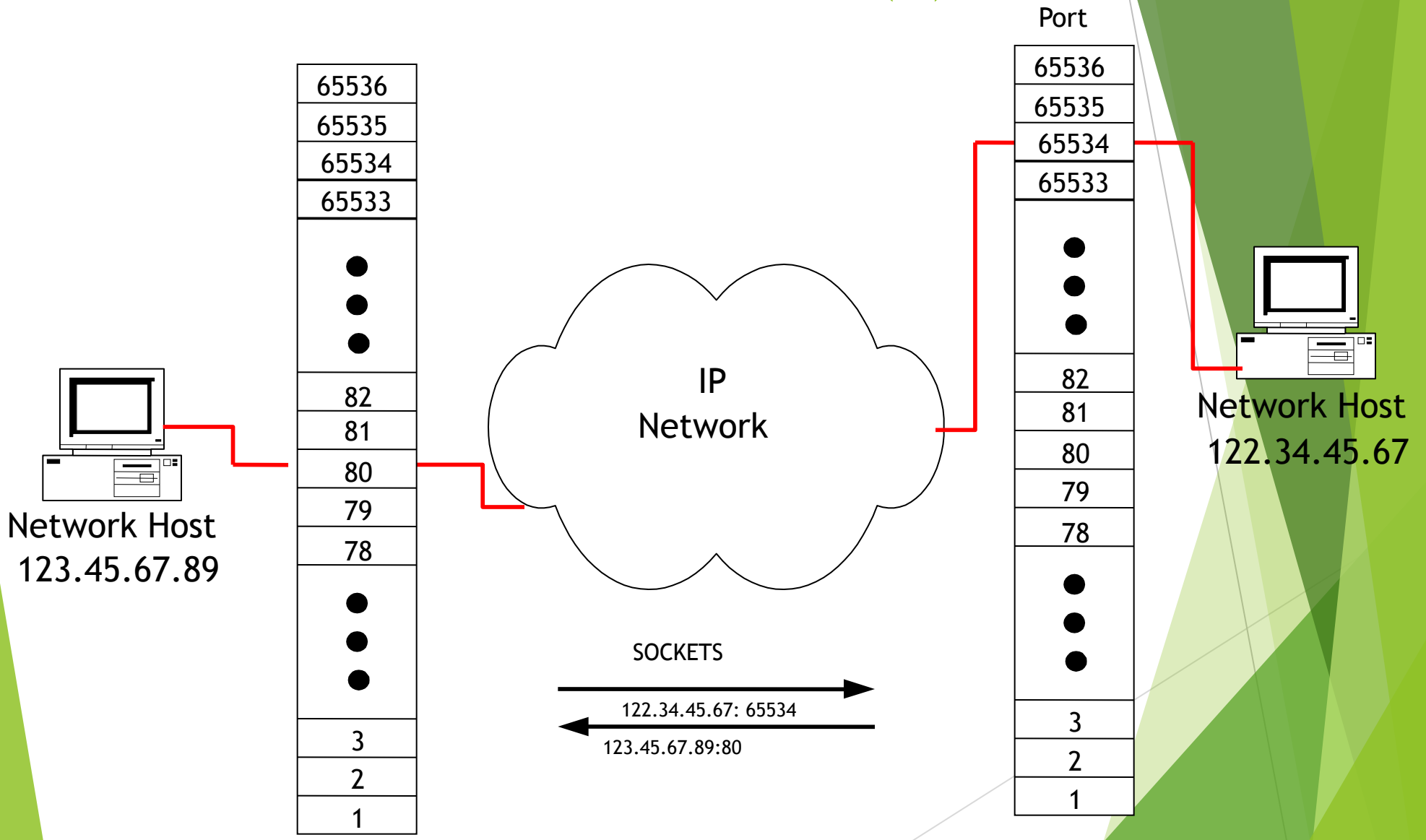
- The transport protocols TCP and UDP were designed to enable communication between network applications

  - Internet host can have several servers running.
    - usually has only one physical link to the "rest of the world"
    - When packets arrive how does the host identify which packets should go to which server?

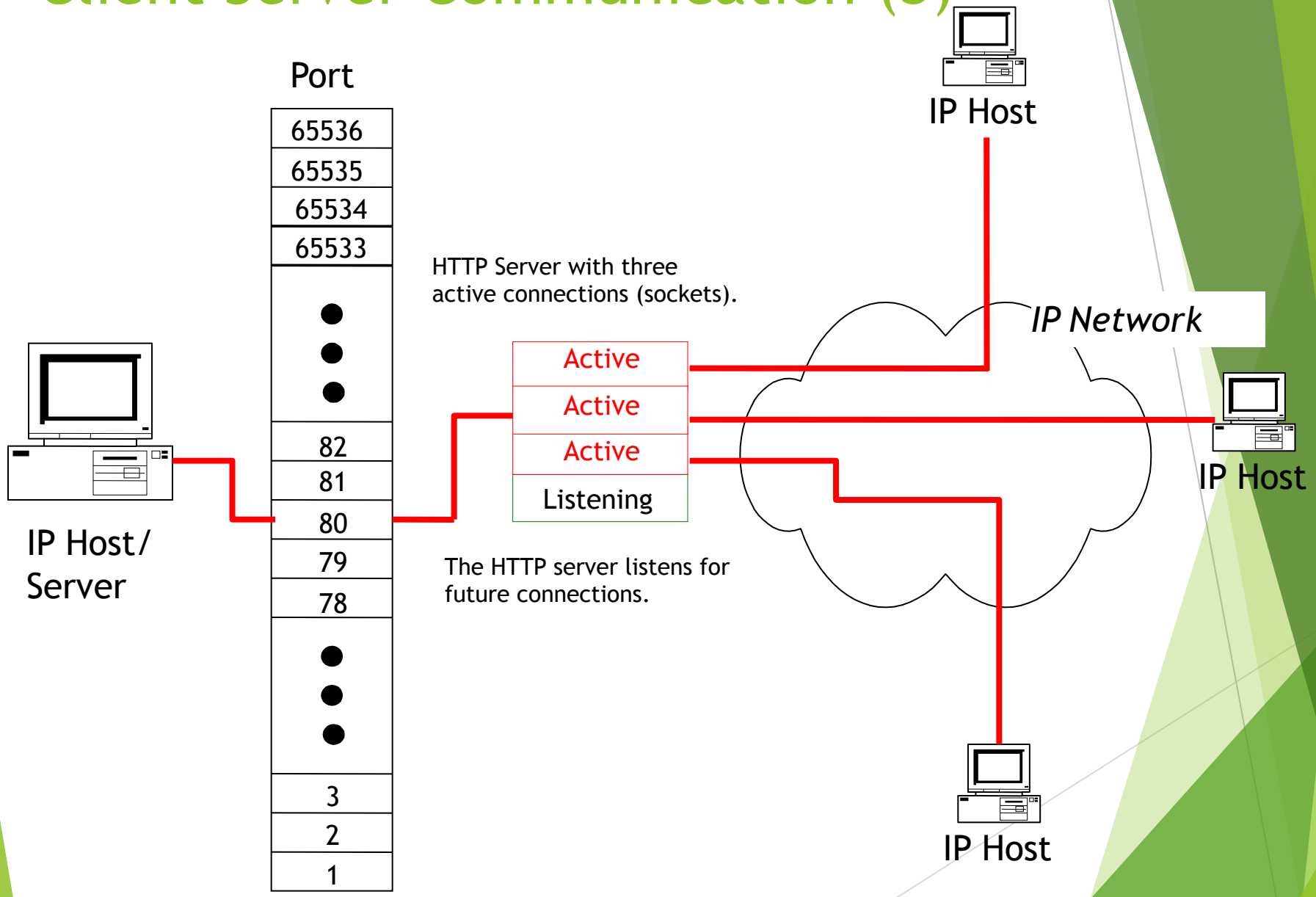
  - Ports
    - ports are used as logical connections between network applications
      - 16 bit number (65536 possible ports)
    - demultiplexing key
      - identify the application/process to receive the packet

  - TCP connection
    - source IP address and source port number
    - destination IP address and destination port number
    - the combination **IP Address : Port Number** pair is called a **Socket**

# Client Server Communication (2)



# Client Server Communication (3)



# Connections

- A *socket address* is the triplet:

  *{protocol, local-IP, local-port}*

  - example,

    *{tcp, 130.245.1.44, 23}*

- A *connection* is the 5-tuple that completely specifies the two end-points that comprise a connection:

  *{protocol, local-IP, local-port, remote-IP, remote-port}*

  - example:

    *{tcp, 130.245.1.44, 23, 130.245.1.45, 1024}*

# Socket Domain Families

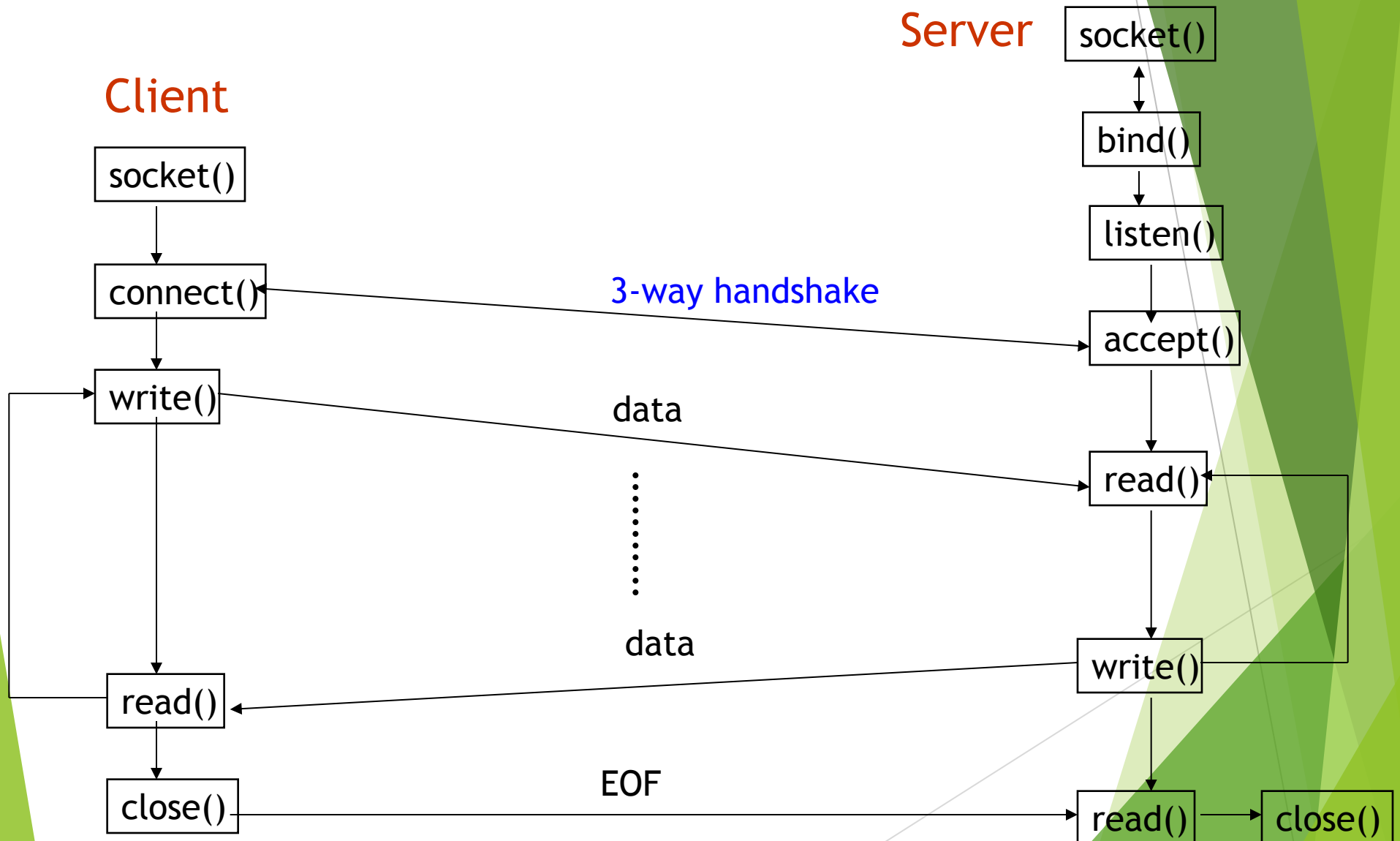
- There are several significant socket domain families:
  - Internet Domain Sockets (AF_INET)
    - implemented via IP addresses and port numbers
  - Unix Domain Sockets (AF_UNIX)
    - implemented via filenames (similar to IPC “named pipe”)

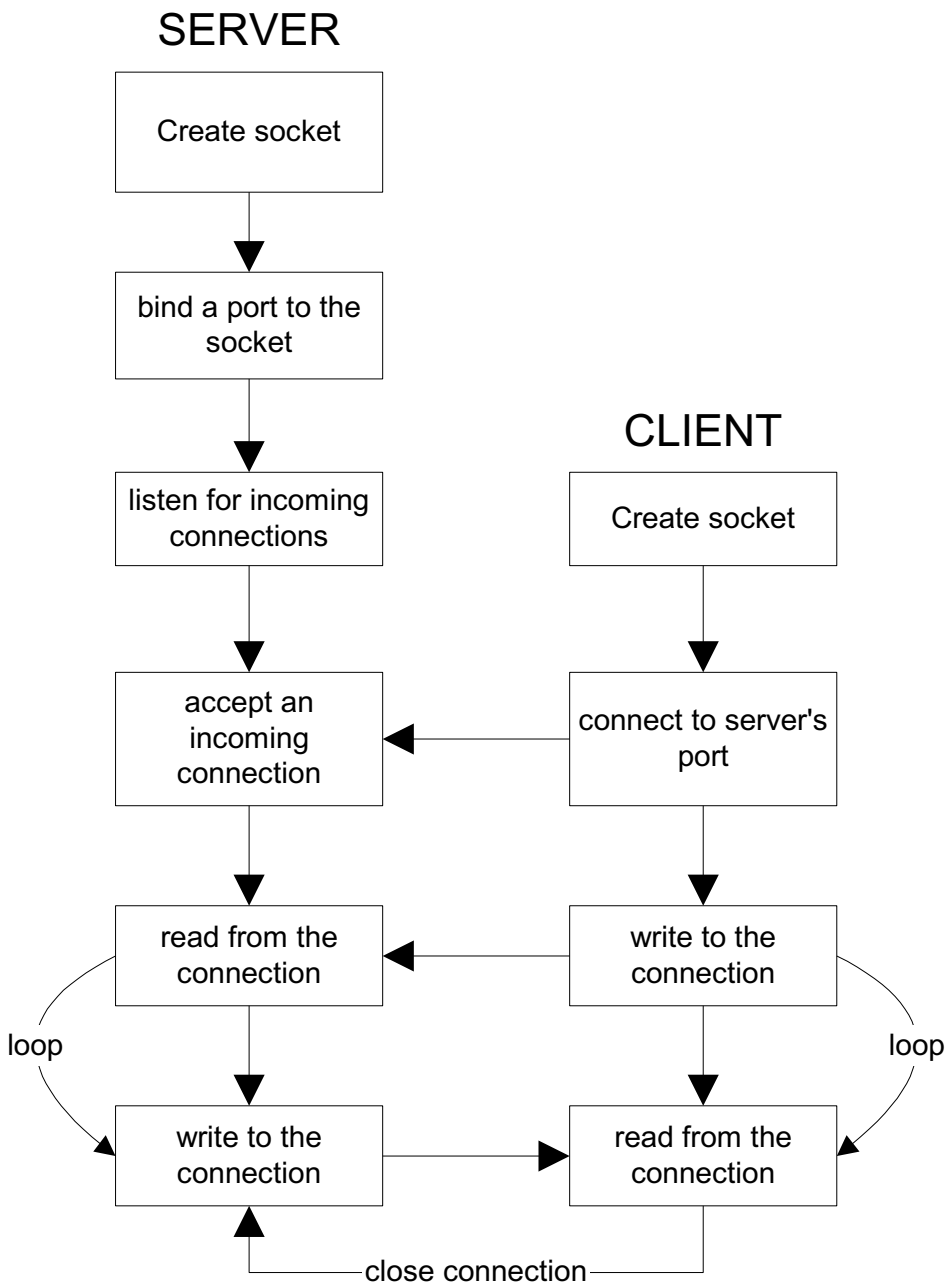
# Creating a Socket

```
#include <sys/types.h>
#include <sys/socket.h>
int socket(int domain, int type, int protocol);
```

- domain is one of the *Protocol Families* (AF_INET, AF_UNIX, etc.)
- type defines the communication protocol semantics, usually defines either:
  - SOCK_STREAM: connection-oriented stream (TCP)
  - SOCK_DGRAM: connectionless, unreliable (UDP)
- protocol specifies a particular protocol, just set this to 0 to accept the default

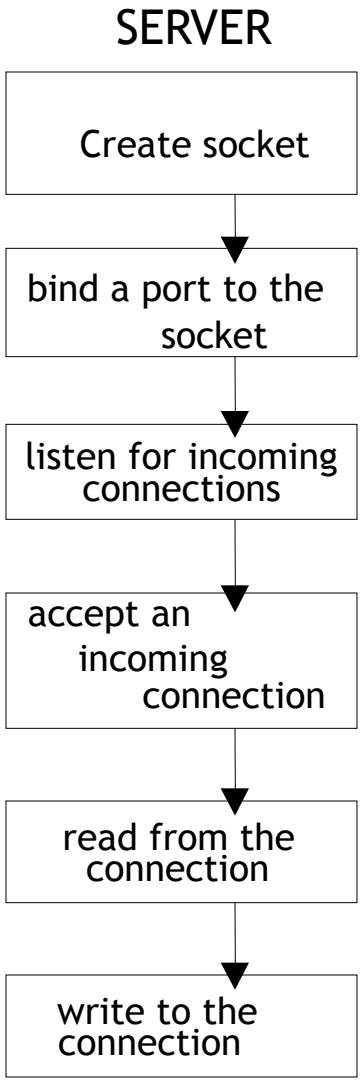
# Stream Socket Transaction (TCP Connection)






- Connection-oriented socket connections
- Client-Server view

# Server-Side Socket Details




int socket(int domain, int type, int protocol)
sockfd = socket(AP_INET, SOCK_STREAM, 0);

int bind(int sockfd, struct sockaddr *server_addr, socklen_t length)
bind(sockfd, &server, sizeof(server));

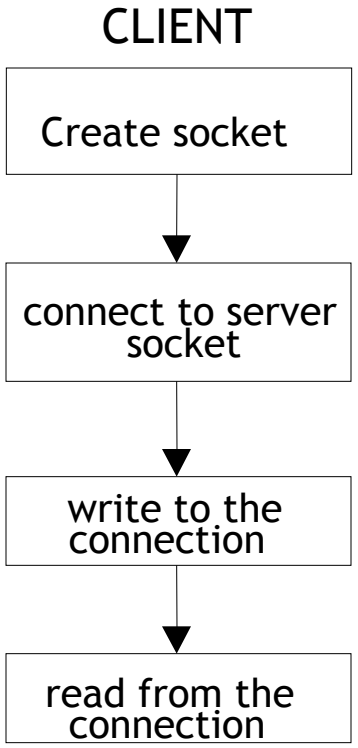
int listen( int sockfd, int num_queued_requests)
listen( sockfd, 5);

int accept(int sockfd, struct sockaddr *incoming_address, socklen_t le
newfd = accept(sockfd, &client, sizeof(client)); /* BLOCKS */

int read(int sockfd, void * buffer, size_t buffer_size)
read(newfd, buffer, sizeof(buffer));

int write(int sockfd, void * buffer, size_t buffer_size)
write(newfd, buffer, sizeof(buffer));

# Client-Side Socket Details




```
int socket(int domain, int type, int protocol)
 sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

```
int connect(int sockfd, struct sockaddr *server_address, socklen_t le
connect(sockfd, &server, sizeof(server));    /* Blocking */
```

```
int write(int sockfd, void * buffer, size_t buffer_size)
write(sockfd, buffer, sizeof(buffer));
```

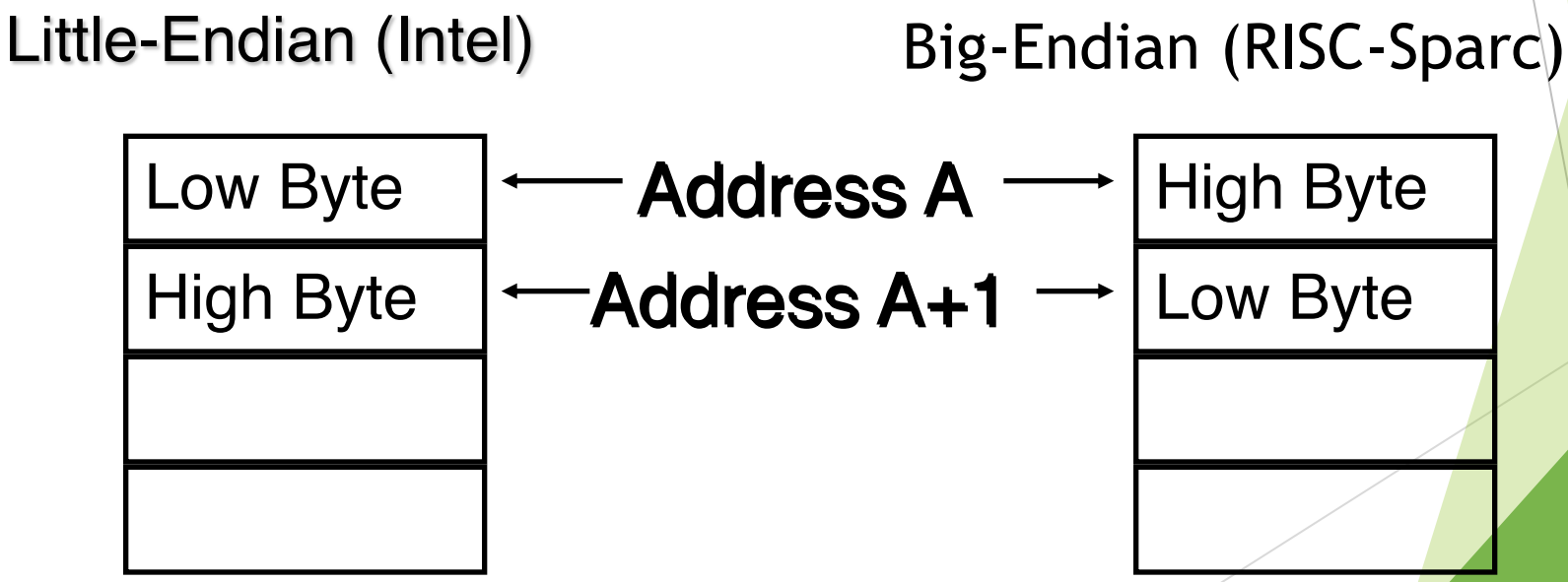
```
int read(int sockfd, void * buffer, size_t buffer_size)
read(sockfd, buffer, sizeof(buffer));
```

# Closing a Socket Session

- int close(int socket);
  - closes read/write IO, closes socket file descriptor
- int shutdown( int socketfd, int mode);
  - where mode is:
    - 0: no more receives allowed "r"
    - 1: no more sends are allowed "w"
    - 2: disables both receives and sends (but doesn't close the socket, use close() for that) "rw"

# Byte Ordering

- Different computer architectures use different byte ordering to represent/store multi-byte values (such as 16-bit/32-bit integers)
- 16 bit integer:



# Byte Order and Networking

- Suppose a Big Endian machine sends a 16 bit integer with the value 2:

```
0000000000000010
```

- A Little Endian machine will understand the number as 512:

```
0000001000000000
```

- How do two machines with different byte-orders communicate?
  - Using network byte-order
  - Network byte-order = big-endian order

- Host byte order
  - Big Endian (IBM mainframes, Sun SPARC) *or*
  - Little Endian (x86)

# Network Byte Order

- Conversion of application-level data is left up to the presentation layer.
- Lower-level layers communicate using a fixed byte order called *network byte order* for all control data.
- TCP/IP mandates that *big-endian* byte ordering be used for transmitting protocol information
- All values stored in a `sockaddr_in` must be in network byte order.
  - `sin_port` a TCP/IP port number.
  - `sin_addr` an IP address.

# Network Byte Order Functions

- Several functions are provided to allow conversion between host and network byte ordering,
- Conversion macros (`<netinet/in.h>`)
  - to translate 32-bit numbers (i.e. IP addresses):
    - unsigned long htonl(unsigned long hostlong);
    - unsigned long ntohl(unsigned long netlong);
  - to translate 16-bit numbers (i.e. Port numbers):
    - unsigned short htons(unsigned short hostshort);
    - unsigned short ntohs(unsigned short netshort);

# Creating a TCP socket

```
int socket(int family,int type,int proto);


int mysockfd;
mysockfd = socket(AF_INET, SOCK_STREAM,0);
if (mysockfd<0) {
    /* ERROR */
   }
```

# Binding to well known address

```
int mysockfd;
int err;
struct sockaddr_in myaddr;

  mysockfd = socket(AF_INET,SOCK_STREAM,0);
   myaddr.sin_family = AF_INET;
   myaddr.sin_port = htons( 80 );
   myaddr.sin_addr = htonl( INADDR_ANY );

  err= bind(mysockfd, (sockaddr *) &myaddr,
              sizeof(myaddr));
```

# Converting Between IP Address formats

- From ASCII to numeric
  - "130.245.1.44" ➔ 32-bit network byte ordered value
  - `inet_aton(…) with IPv4`
  - `inet_pton(…) with IPv4 and IPv6`
- From numeric to ASCII
  - 32-bit value ➔ "130.245.1.44"
  - `inet_ntoa(…) with IPv4`
  - `inet_ntop(…) with IPv4 and IPv6`
  - Note - `inet_addr(…)` obsolete
    - cannot handle broadcast address "255.255.255.255" (0xFFFFFFFF)

# IPv4 Address Conversion

```
int inet_aton( char *, struct in_addr *);
```

Convert ASCII dotted-decimal IP address to network byte order 32 bit value. Returns 1 on success, 0 on failure.

```
char *inet_ntoa(struct in_addr);
```

Convert network byte ordered value to ASCII dotted-decimal (a string).

# Establishing a passive mode TCP socket

- Passive mode:
  - Address already determined.
  - Tell the kernel to accept incoming connection requests directed at the socket address.
    - ***3-way handshake***
  - Tell the kernel to queue incoming connections for us.

# listen()

```
int listen( int mysockfd, int backlog);
```

`mysockfd` is the TCP socket (already bound to an address)

`backlog` is the number of incoming connections the kernel should be able to keep track of (queue for us).

`listen()` returns -1 on error (otherwise 0).

# Accepting an incoming connection

- Once we call `listen()`, the O.S. will queue incoming connections
  - Handles the 3-way handshake
  - Queues up multiple connections.
- When our application is ready to handle a new connection, we need to ask the O.S. for the next connection.

# accept()

```
int accept( int mysockfd,
            struct sockaddr* cliaddr,
            socklen_t *addrlen);
```

`mysockfd` is the passive mode TCP socket.

`cliaddr` is a pointer to *allocated* space.

`addrlen` is a *value-result* argument

- must be set to the size of `cliaddr`
- on return, will be set to be the number of used bytes in `cliaddr`.

- `accept()` return value
  - `accept()` returns a new socket descriptor (positive integer) or -1 on error.
  - After `accept` returns a new socket descriptor, I/O can be done using the `read()` and `write()` system calls.

# Terminating a TCP connection

- Either end of the connection can call the `close()` system call.
- If the other end has closed the connection, and there is no buffered data, reading from a TCP socket returns 0 to indicate EOF.

# Client Code

- TCP clients can call `connect()` which:
  - takes care of establishing an endpoint address for the client socket.
    - don't need to call bind first, the O.S. will take care of assigning the local endpoint address (TCP port number, IP address).
  - Attempts to establish a connection to the specified server.
    - ***3-way handshake***

# connect()

```
int connect( int sockfd,
             const struct sockaddr *server,
             socklen_t addrlen);
```

`sockfd` is an already created TCP socket.

`server` contains the address of the server (IP Address and TCP port number)

`connect()` returns 0 if OK, -1 on error

# Reading from a TCP socket

```
int read( int fd, char *buf, int max);
```

- By default `read()` will block until data is available.
- reading from a TCP socket may return less than max bytes (whatever is available).

# Writing to a TCP socket

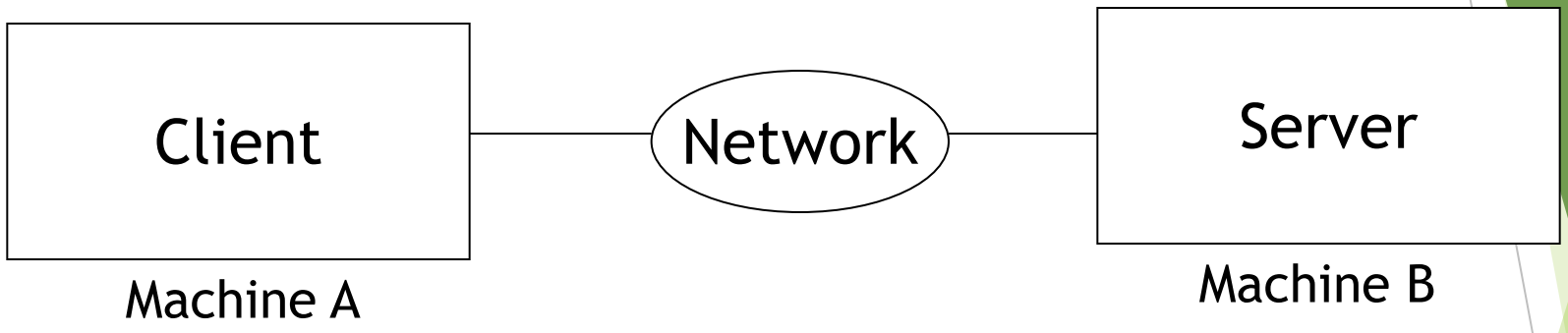
```
int write( int fd, char *buf, int num);
```

- write might not be able to write all num bytes (on a nonblocking socket).

- Other functions (API)
  - readn(), writen() and readline() - see man pages definitions.

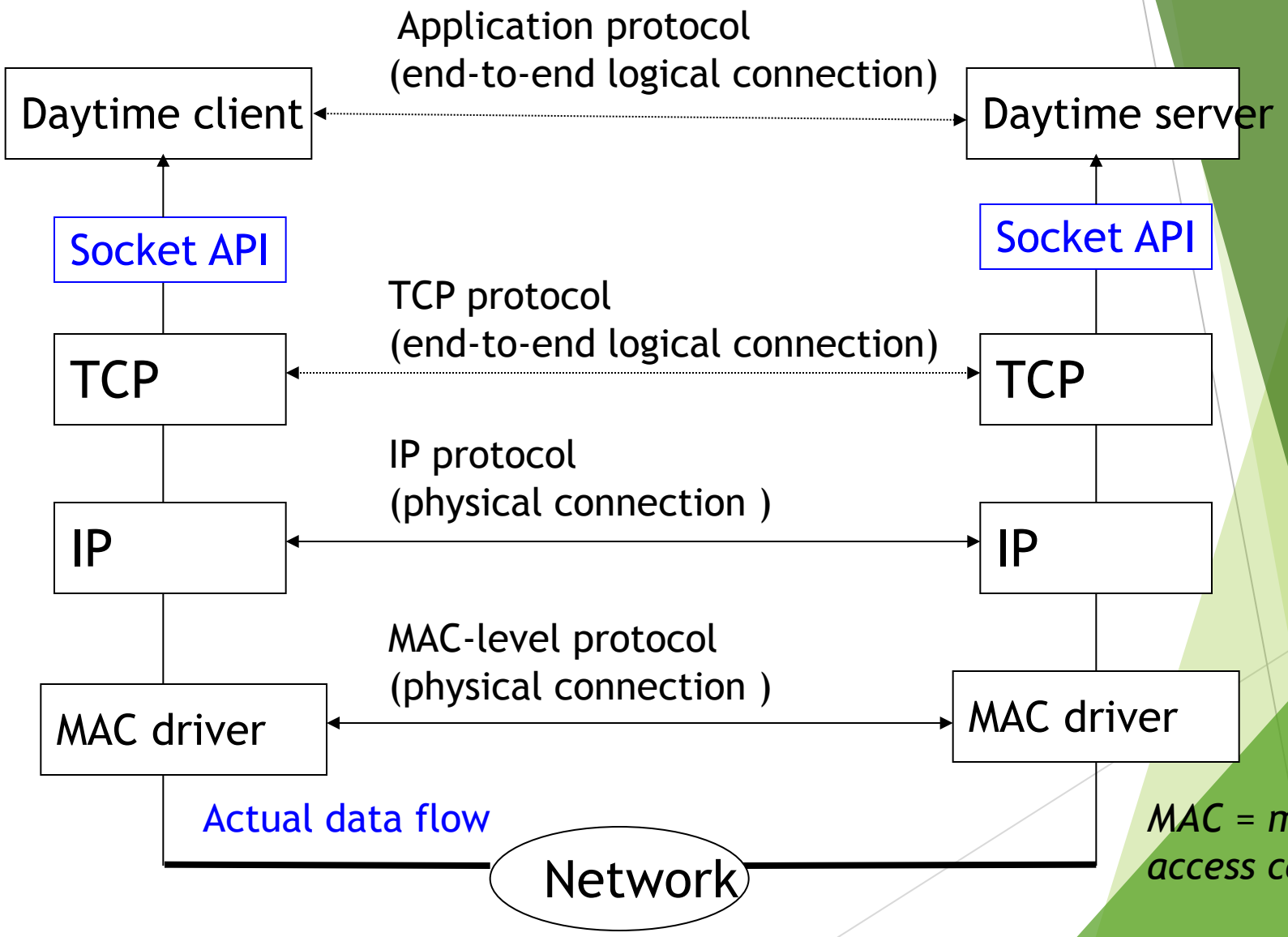
# Example

Client Server communication




- Web browser and server
- FTP client and server
- Telnet client and server

# Example – Daytime Server/Client



# Daytime client

- **Connects to a daytime server**
- **Retrieves the current date and time**

```
% gettime  130.245.1.44
Thu Sept 05 15:50:00 2002
```

```
#include        "unp.h"
int main(int argc, char **argv)
{
        int  sockfd, n;
        char recvline[MAXLINE + 1];
        struct sockaddr_in servaddr;

       if( argc != 2 )err_quit("usage : gettime <IP address>");

        /* Create a TCP socket */
        if ( (sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
                err_sys("socket error");


        /* Specify server's IP address and port */
        bzero(&servaddr, sizeof(servaddr));
        servaddr.sin_family = AF_INET;
        servaddr.sin_port = htons(13); /* daytime server port */
        if (inet_pton(AF_INET, argv[1], &servaddr.sin_addr) <= 0)
                err_quit("inet_pton error for %s", argv[1]);
```

# Daytime client

```
    /* Connect to the server */
  if (connect(sockfd, (SA *) &servaddr, sizeof(servaddr)) < 0)
          err_sys("connect error");

    /* Read the date/time from socket */
    while ( (n = read(sockfd, recvline, MAXLINE)) > 0) {
          recvline[n] = 0;          /* null terminate */
          printf("%s", recvline);
    }

   if (n < 0) err_sys("read error");

    close(sockfd);
}
```

# Daytime Server

- Waits for requests from Client
- Accepts client connections
- Send the current time
- Terminates connection and goes back waiting for more connections.

```
#include        "unp.h"
#include        <time.h>

int main(int argc, char **argv)
{
        int    listenfd, connfd;
        struct sockaddr_in servaddr;
        char buff[MAXLINE];
        time_t ticks;

        /* Create a TCP socket */
        listenfd = socket(AF_INET, SOCK_STREAM, 0);

        /* Initialize server's address and well-known port */
        bzero(&servaddr, sizeof(servaddr));
        servaddr.sin_family      = AF_INET;
        servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
        servaddr.sin_port        = htons(13);   /* daytime server */

        /* Bind server's address and port to the socket */
        Bind(listenfd, (SA *) &servaddr, sizeof(servaddr));
```

# Daytime Server

```
 /* Convert socket to a listening socket */
 listen(listenfd, LISTENQ);

for ( ; ; ) {
 /* Wait for client connections and accept them */
 connfd = accept(listenfd, (SA *) NULL, NULL);

 /* Retrieve system time */
 ticks = time(NULL);
 sprintf(buff, sizeof(buff), "%.24s\r\n", ctime(&ticks));

  /* Write to socket */
  write(connfd, buff, strlen(buff));

/* Close the connection */
  close(connfd);
      }
}
```